

Sala de Emergencias

Diseño e implementación de estructuras lineales y su aplicación a un sistema de gestión hospitalaria

EQUIPO 3

Federico Martínez · Milagros García · Santiago Acosta · Alexis Gimenez · Bruno Pereda

AED · 2026

El proyecto en tres desafíos

DESAFÍO 1

Construir la caja de herramientas

Ocho estructuras lineales implementadas a partir de sus TDA, con pruebas para cada una.

DESAFÍO 2

Modelar la sala de emergencias

Representar el flujo de atención usando exclusivamente las estructuras propias y justificar cada elección.

DESAFÍO 3

Optimizar una operación crítica

Identificar un cuello de botella, proponer otra representación, implementarla y comparar.

01

Modelado del dominio

Entidades del modelo

Paciente

ID y nombre, una lista de características, un nivel de urgencia y un estado dentro del proceso.

Consulta

ID del paciente, urgencia, procedimiento realizado y fecha. Es el registro histórico de una atención.

NivelUrgencia · EstadoPaciente

Dos enums separados: la prioridad de atención y la etapa del proceso son conceptos distintos.

SalaEmergencia

Coordina las cuatro estructuras: registro, espera, consultorios e historial.

Niveles de urgencia

Cada nivel lleva asociado el tiempo máximo de espera tolerable en minutos. Ese valor, y no el orden de llegada, define la posición en la cola.

NIVEL	ESPERA MÁXIMA TOLERABLE	POSICIÓN EN LA COLA
CRITICO	0 minutos	Primero
URGENTE	10 minutos	—
MODERADO	60 minutos	—
LEVE	120 minutos	—
NO_URGENTE	240 minutos	Último

Estados del paciente

REGISTRADO

Ingresa al registro general.
Todavía sin urgencia asignada.

EN_ESPERA

Evaluated and placed in the queue
of priority according to its urgency.

EN_CONSULTORIO

Ocupa una de las posiciones de
la lista de consultorios.

ATENDIDO

Dado de alta. Su consulta queda
apilada en el historial.

La urgencia describe con qué prioridad debe atenderse; el estado, en qué etapa del proceso se encuentra. Mantenerlos separados evita mezclar dos características distintas del dominio.

Estructuras utilizadas

Una estructura por necesidad

REGISTRO GENERAL DE PACIENTES

ListaArray

Ordenada por ID. Acceso por índice en $O(1)$ habilita la búsqueda binaria.

ESPERA DE ATENCIÓN

ColaPrioridad

FIFO no representa el problema: el orden lo determina la urgencia.

HISTORIAL DE CONSULTAS

Pila

LIFO: la atención más reciente queda disponible primero, en $O(1)$.

CARACTERÍSTICAS DEL PACIENTE

ListaEnlazada

Colección pequeña: solo se agrega y se elimina, sin acceso por posición.

Incorporamos los consultorios como etapa del proceso

Técnicamente podíamos retirar al paciente de la cola y registrar su consulta al terminar. Agregamos los consultorios para demostrar un manejo más completo de las estructuras: una segunda `ListaArray` gestionada por posición, con capacidad limitada y ampliable.

Modela una etapa intermedia real: el paciente deja la espera antes de quedar atendido.

Obliga a controlar un recurso limitado: verificar capacidad antes de ingresar.

Asocia cada paciente a un número de consultorio concreto mediante el índice.

Los consultorios como posiciones

Se representan con una **ListaArray** de capacidad cinco: cada índice corresponde a un consultorio concreto, y verificar si hay lugar es comparar tamaño con capacidad.

Consultorio 0 Ocupado	Consultorio 1 Ocupado	Consultorio 2 Ocupado	Consultorio 3 Libre	Consultorio 4 Libre
---------------------------------	---------------------------------	---------------------------------	-------------------------------	-------------------------------

La operación `agregarConsultorio()` amplía la capacidad, modelando una ampliación real de los recursos de la sala.

La prioridad vive en la inserción

```
@Override
public boolean poneEnCola(T dato) {
    Nodo<T> actual = cabeza;
    int contador = 0;
    while (actual != null
           && comparador.compare(
               dato, actual.getDato()) >= 0) {
        contador++;
        actual = actual.siguiente;
    }
    super.agregar(contador, dato);
    return true;
}
```

Criterio de orden

El comparador POR_URGENCIA compara el tiempo máximo tolerable de cada paciente.

Estabilidad

Con ≥ 0 el nuevo paciente queda detrás de los de igual urgencia: se respeta el orden de llegada.

Invariante

La inserción por índice se deshabilita: la posición la determina la prioridad, nunca quien llama.

Búsqueda binaria sobre el registro

```
private int indiceBinario(String idPaciente) {  
    int lo = 0, hi = pacientesRegistrados.tamaño() - 1;  
    while (lo <= hi) {  
        int medio = (lo + hi) / 2;  
        int cmp = pacientesRegistrados.obtener(medio)  
            .getId().compareTo(idPaciente);  
  
        if (cmp == 0) return medio;  
        if (cmp < 0) lo = medio + 1;  
        else hi = medio - 1;  
    }  
    return -(lo + 1); // no está: punto de inserción  
}
```

Requiere acceso $O(1)$

Solo es viable sobre ListaArray: sobre una lista enlazada, cada obtener costaría $O(n)$.

Doble uso

Un valor negativo indica que el ID no existe y codifica dónde debe insertarse para mantener el orden.

Resultado

Buscar un paciente entre n registrados pasa a costar $O(\log n)$.

Operaciones principales

registrarPaciente

Verifica que el ID no exista y lo inserta en su posición ordenada.

agregarPacienteACola

Asigna el nivel de urgencia y lo incorpora a la espera según prioridad.

ingresarPaciente

Lo retira de la cola y lo ubica en un consultorio libre.

darDeAlta

Libera el consultorio y apila la consulta con el procedimiento realizado.

eliminarPaciente

Lo retira del registro activo, la espera y los consultorios; el historial se conserva.

A estas se suman las consultas de información: `buscarPaciente`, `listarPacientes`, `listarConsultas` y `mostrarPacientesEnConsultorios`.

Complejidad y optimización

Complejidad temporal

OPERACIÓN	COSTO	POR QUÉ
buscarPaciente	$O(\log n)$	Búsqueda binaria sobre la ListaArray ordenada por ID.
registrarPaciente	$O(n)$	Localiza en $O(\log n)$, pero insertar desplaza los elementos posteriores.
agregarPacienteACola	$O(n)$	La inserción por prioridad recorre la cola hasta su posición.
ingresarPaciente	$O(n)$	Domina el retiro de la cola; ubicarlo en consultorio es $O(1)$.
darDeAlta	$O(1)$	Acceso por posición en el array y apilado de la consulta.
eliminarPaciente	$O(n)$	Debe retirarlo de la espera, los consultorios y el registro.

Desafío 3: el registro de pacientes

ANTES

ListaEnlazada

Búsqueda

Inserción

$O(n)$ **$O(1)$**

Cada búsqueda recorría los nodos desde la cabeza, y la búsqueda se invocaba desde registrar, encolar y eliminar.

DESPUÉS

ListaArray ordenada

Búsqueda

Inserción

$O(\log n)$ **$O(n)$**

El acceso por índice en $O(1)$ habilita la búsqueda binaria; mantener el orden exige desplazar elementos al insertar.

Se registra una vez por paciente, pero se busca muchas veces y desde varias operaciones: el intercambio favorece a la operación más frecuente.

Video de la demostración

Conclusiones

Linked Lists be like

